

LAPORAN PRATIUM PEMOGRAMAN BERORIENTASI OBJEK

“RELASI ANTAR OBJEK”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH PEMOGRAMAN BERORIENTASI
OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM.
ASISTEN PRAKTIKUM ; EZZA ADDINI

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

A. Langkah Kerja Praktikum

1. Membuat Class Transaksi

- Membuat class baru dengan nama Transaksi.

```
public class Transaksi {
```

- Menambahkan atribut idTransaksi, jenis, dan nominal.

```
String idTransaksi;  
String jenis;  
double nominal;
```

- Membuat constructor untuk mengisi ketiga atribut tersebut.

```
// Constructor  
public Transaksi(String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

- Membuat method cetakDetail() untuk menampilkan detail transaksi.

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);  
}
```

2. Menambahkan Relasi pada Class Rekening

- Membuka kembali class Rekening yang telah dibuat pada praktikum sebelumnya.
- Menambahkan atribut riwayatTransaksi dengan tipe ArrayList<Transaksi>.

```
// Implementasi Asosiasi (1-to-many)  
ArrayList<Transaksi> riwayatTransaksi;
```

- Menginisialisasi riwayatTransaksi pada constructor.

```
this.riwayatTransaksi = new ArrayList<>();
```

- ArrayList digunakan untuk menyimpan kumpulan objek Transaksi yang dimiliki oleh suatu rekening.
- Dengan penambahan tersebut, satu objek Rekening dapat memiliki banyak objek Transaksi.

3. Menambahkan Pencatatan Transaksi pada Setor Tunai

- Membuka method setorTunai(double nominal).
- Membuat objek Transaksi baru dengan jenis Kredit.

```
Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
```

- Membuat ID transaksi dengan awalan TRX-S-.

```
String idTrx = "TRX-S-" + System.currentTimeMillis();
```

- Menambahkan objek transaksi ke dalam riwayatTransaksi.

```
riwayatTransaksi.add(trxBaru);
```

B. Latihan / Tugas Praktikum

a. Integrasi Rekam Jejak Penarikan

1. Membuka method tarikTunai(double nominal) pada class Rekening.
2. Menambahkan pembuatan objek Transaksi setelah penarikan berhasil
3. Mengatur jenis transaksi menjadi Debit.

```
Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
```

4. Membuat ID transaksi dengan awalan TRX-T-.

```
String idTrx = "TRX-T-" + System.currentTimeMillis();
```

5. Menambahkan objek transaksi ke dalam riwayatTransaksi.

```
riwayatTransaksi.add(trxBaru);
```

b. Fitur Cetak Mutasi Rekening

1. Membuat method cetakMutasi() pada class Rekening.

```
public void cetakMutasi() {
```

2. Mengecek apakah riwayatTransaksi kosong.

```
if (riwayatTransaksi.isEmpty()) {
```

3. Jika kosong, menampilkan pesan Belum ada transaksi pada rekening ini.

4. Jika terdapat transaksi, menggunakan perulangan for-each untuk membaca seluruh transaksi.

5. Memanggil method cetakDetail() dari setiap objek Transaksi.

```
for (Transaksi trx : riwayatTransaksi) {  
    trx.cetakDetail();
```

6. Membuka class Main.

7. Menambahkan menu 6. Cetak Mutasi (Riwayat).

```
System.out.println("6. Cetak Mutasi(Riwayat)");
```

8. Memanggil method cetakMutasi() dari akunAktif ketika menu 6 dipilih.

```
akunAktif.cetakMutasi();
```

C. Hasil Kode Program

```
Menu Utama:  
1. Buka Rekening Baru  
2. Setor Tunai  
3. Tarik Tunai  
4. Cek Informasi Rekening  
5. Ganti Akun  
6. Cetak Mutasi(Riwayat)  
7. Ringkasan Transaksi (challenge  
0. Keluar  
Pilih menu: 6  
=== MUTASI REKENING 2511531007 ===  
ID: TRX-S-1790087102904 | Jenis: Kredit | Nominal: Rp50000.0  
ID: TRX-T-1790087115203 | Jenis: Debit | Nominal: Rp25000.0
```